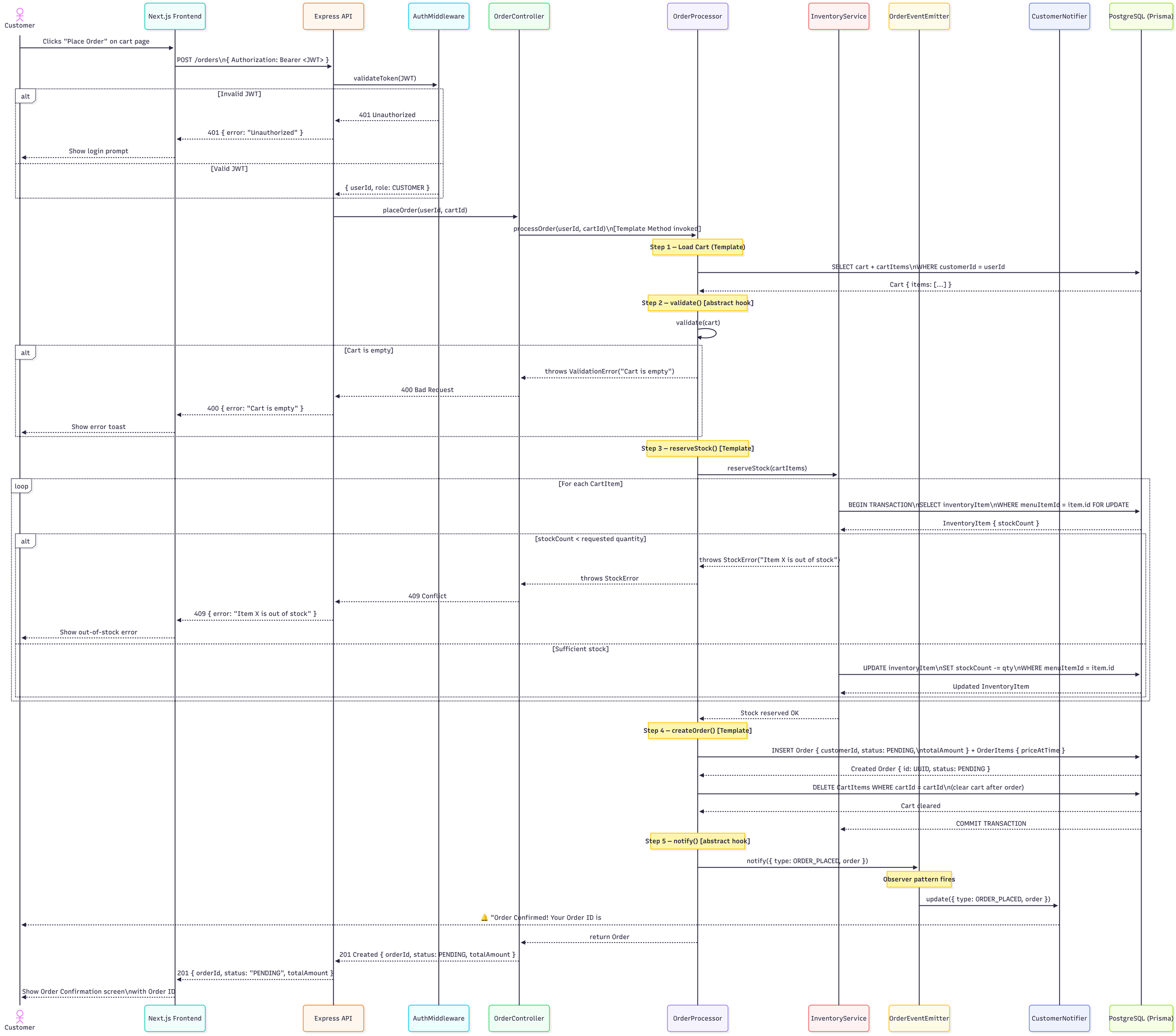
Customer
Next.js Frontend
Express API
AuthMiddleware
OrderController
OrderProcessor
InventoryService
OrderEventEmitter
CustomerNotifier
PostgreSQL (Prisma)
Clicks "Place Order" on cart page
POST /orders\n{ Authorization: Bearer <JWT> }
validateToken(JWT)
alt
[Invalid JWT]
401 Unauthorized
401 { error: "Unauthorized" }
Show login prompt
[Valid JWT]
{ userId, role: CUSTOMER }
placeOrder(userId, cartId)
processOrder(userId, cartId)\n[Template Method invoked]
Step 1 — Load Cart (Template)
SELECT cart + cartItems\nWHERE customerId = userId
Cart { items: [...] }
Step 2 — validate() [abstract hook]
validate(cart)
alt
[Cart is empty]
throws ValidationError("Cart is empty")
400 Bad Request
400 { error: "Cart is empty" }
Show error toast
Step 3 — reserveStock() [Template]
reserveStock(cartItems)
loop
[For each CartItem]
BEGIN TRANSACTION\nSELECT inventoryItem\nWHERE menuItemId = item.id FOR UPDATE
InventoryItem { stockCount }
alt
[stockCount < requested quantity]
throws StockError("Item X is out of stock")
throws StockError
409 Conflict
409 { error: "Item X is out of stock" }
Show out-of-stock error
[Sufficient stock]
UPDATE inventoryItem\nSET stockCount -= qty\nWHERE menuItemId = item.id
Updated InventoryItem
Stock reserved OK
Step 4 — createOrder() [Template]
INSERT Order { customerId, status: PENDING,\ntotalAmount } + OrderItems { priceAtTime }
Created Order { id: UUID, status: PENDING }
DELETE CartItems WHERE cartId = cartId\n(clear cart after order)
Cart cleared
COMMIT TRANSACTION
Step 5 — notify() [abstract hook]
notify({ type: ORDER_PLACED, order })
Observer pattern fires
update({ type: ORDER_PLACED, order })
🔔 "Order Confirmed! Your Order ID is
return Order
201 Created { orderId, status: PENDING, totalAmount }
201 { orderId, status: "PENDING", totalAmount }
Show Order Confirmation screen\nwith Order ID